

LoginSystem

Version 1.0

Secure Multi-Step Authentication System

Technical Documentation

Attribute	Value
Project Version	1.0.0
Framework	Django 5.x / Python 3.11+
Architecture	Model-View-Template (MVT)
Database	SQLite / PostgreSQL Compatible

Table of Contents

Table of Contents	2
1. System Architecture Overview	3
State Transition Diagram	3
2. Database Schema (models.py)	3
Field Definitions	3
Schema Mapping Matrix	4
3. Route Configuration (urls.py)	4
Main Router — config/urls.py	4
App-Level Router — secure_auth/urls.py	4
4. Business Logic Layer (views.py)	5
4.1 Step 1 — Initial Registration (signup_step1_view)	5
Workflow Summary	6
4.2 Step 2 — OTP Verification (verify_otp_view)	6
Workflow Summary	6
4.3 Step 3 — Account Activation (activate_password_view)	6
Workflow Summary	7
5. Session Management & Middleware Flow	7
6. Security Guarantees	8
7. Production Hardening Notes	8

1. System Architecture Overview

LoginSystem implements a structured three-step onboarding pipeline for new users. Rather than collecting credentials in a single form, the process is split into distinct stages to guarantee email ownership before an account is ever activated.

Step 1: Registration (First/Last Name, Email)	--->	Step 2: OTP Verification (6-Digit Email Code)	--->	Step 3: Password & Activation (Password Hash Creation)
--	------	--	------	---

State Transition Diagram



2. Database Schema (models.py)

The system relies on a single unified model, `UserAccount`, inheriting from `django.db.models.Model`. Data integrity is enforced through database constraints (`unique=True`) alongside boolean state-tracking flags.

Field Definitions

```

from django.db import models

class UserAccount(models.Model):
    # 1. Core Profile Details (Collected in Step 1)
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    email = models.EmailField(unique=True)

    # 2. Temporary Security Code
    otp_code = models.CharField(max_length=6, blank=True, null=True)
  
```

```
# 3. Permanent Security Credentials
password_hash = models.CharField(max_length=255, blank=True, null=True)

# 4. Operational State Flags
email_verified = models.BooleanField(default=False)
active_status = models.BooleanField(default=False)

def __str__(self):
    return f"{self.first_name} {self.last_name} ({self.email}) - Active: {self.active_status}"
```

Schema Mapping Matrix

Field Name	Type	Constraints / Defaults	Purpose
id	BigAutoField	Auto Primary Key	Internal row identifier
first_name	CharField	max_length=50	User's given name
last_name	CharField	max_length=50	User's family name
email	EmailField	unique=True	Primary identifier & email target
otp_code	CharField	max_length=6, blank=True, null=True	Temporary verification token
password_hash	CharField	max_length=255, blank=True, null=True	Hashed user password
email_verified	BooleanField	default=False	Tracks successful OTP validation
active_status	BooleanField	default=False	Tracks full account activation

3. Route Configuration (urls.py)

Routing uses Django's reverse resolution pattern (the `name=` attribute) to decouple URL paths from application view logic.

Main Router — `config/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('secure_auth.urls')), # Relays remaining paths to app router
]
```

App-Level Router — `secure_auth/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('signup/', views.signup_step1_view, name='signup_step1'),
```

```

    path('verify-otp/', views.verify_otp_view, name='verify_otp'),
    path('activate-account/', views.activate_password_view, name='activate_password'),
]

```

Path	View	Name
/signup/	signup_step1_view	signup_step1
/verify-otp/	verify_otp_view	verify_otp
/activate-account/	activate_password_view	activate_password

4. Business Logic Layer (views.py)

4.1 Step 1 — Initial Registration (signup_step1_view)

Handles profile data collection, stale-record purging, dynamic OTP creation, and SMTP dispatch.

```

import random
from django.shortcuts import render, redirect
from django.core.mail import send_mail
from django.conf import settings
from .models import UserAccount

def signup_step1_view(request):
    if request.method == "POST":
        fname = request.POST.get("first_name")
        lname = request.POST.get("last_name")
        user_email = request.POST.get("email")

        # Security Guardrail: Reject if email belongs to an already active user
        if UserAccount.objects.filter(email=user_email, active_status=True).exists():
            return render(request, "secure_auth/signup_step1.html", {
                "error_message": "This email address is already registered.",
                "typed_fname": fname,
                "typed_lname": lname
            })

        # Cleanup Routine: Remove unactivated/stale sessions using this email
        UserAccount.objects.filter(email=user_email, active_status=False).delete()

        # Token Generation
        generated_otp = str(random.randint(100000, 999999))

        # Database Insertion
        user_record = UserAccount.objects.create(
            first_name=fname,
            last_name=lname,
            email=user_email,
            otp_code=generated_otp
        )

        # Email Dispatch
        send_mail(
            subject="Your Verification OTP",
            message=f"Hello {fname},\n\nYour code is: {generated_otp}",
            from_email=settings.DEFAULT_FROM_EMAIL,
            recipient_list=[user_email],
            fail_silently=False,
        )

```

```
# Session Persistence
request.session["verifying_email"] = user_email

return redirect("verify_otp")

return render(request, "secure_auth/signup_step1.html")
```

Workflow Summary

- Rejects the request if the email already belongs to an active account.
- Deletes any incomplete/stale record left behind by a previous, abandoned attempt with the same email.
- Generates a 6-digit numeric OTP and stores it against a new UserAccount row.
- Sends the OTP via email and stashes verifying_email in the session for the next step.

4.2 Step 2 — OTP Verification (verify_otp_view)

Reads state from SessionMiddleware, compares the user's input to the stored database record, and marks email_verified = True on success.

```
def verify_otp_view(request):
    user_email = request.session.get("verifying_email")

    # Gatekeeper: Block direct URL access without prior session
    if not user_email:
        return redirect("signup_step1")

    error_message = None

    if request.method == "POST":
        user_input_otp = request.POST.get("otp")

        try:
            account = UserAccount.objects.get(email=user_email)

            if account.otp_code == user_input_otp:
                account.email_verified = True
                account.save()
                return redirect("activate_password")
            else:
                error_message = "The OTP didn't match."

        except UserAccount.DoesNotExist:
            return redirect("signup_step1")

    return render(request, "secure_auth/verify_otp.html", {"error_message": error_message})
```

Workflow Summary

- Refuses to render unless a valid verifying_email session key exists.
- Compares the submitted OTP against the value stored on the UserAccount row.
- On match, flips email_verified and advances to Step 3; otherwise surfaces an inline error.

4.3 Step 3 — Account Activation (activate_password_view)

Collects the user's password, activates the account, clears temporary session flags, and completes onboarding.

```
def activate_password_view(request):
    user_email = request.session.get("verifying_email")

    if not user_email:
        return redirect("signup_step1")

    if request.method == "POST":
        raw_password = request.POST.get("password")

        try:
            account = UserAccount.objects.get(email=user_email)

            account.password_hash = raw_password # Note: Swap with make_password() in production
            account.active_status = True
            account.save()

            # Session State Purge
            del request.session["verifying_email"]

            return render(request, "secure_auth/success.html", {"first_name":
account.first_name})

        except UserAccount.DoesNotExist:
            return redirect("signup_step1")

    return render(request, "secure_auth/activate_password.html")
```

Workflow Summary

- Requires an active verifying_email session, same as Step 2.
- Persists the submitted password and flips active_status to True.
- Deletes the verifying_email session key so the flow cannot be replayed.
- Renders a success page confirming activation.

5. Session Management & Middleware Flow

[Browser]	[Django Backend]
--- 1. POST /signup/ ----->	Creates User row & saves "verifying_email"
<-- 2. HTTP 302 (Set-Cookie: sessionid=XYZ) -----	Stores session data in django_session table
--- 3. POST /verify-otp/ (Cookie: sessionid=XYZ) --->	SessionMiddleware resolves sessionid=XYZ, loads "verifying_email" into request.session
<-- 4. HTTP 302 -----	
--- 5. POST /activate-account/ (Cookie=XYZ) ----->	Verifies session, sets active_status=True, deletes "verifying_email"
<-- 6. HTTP 200 OK (Render Success) -----	

Step	Request	Backend Action
1	POST /signup/	Creates UserAccount, saves verifying_email
2	302 Redirect	Sets sessionid cookie, persists session row

Step	Request	Backend Action
3	POST /verify-otp/	Loads verifying_email from session, validates OTP
4	302 Redirect	Advances to activation step
5	POST /activate-account/	Sets active_status=True, purges session key
6	200 OK	Renders success page

6. Security Guarantees

Guarantee	Description
Anti-Account Enumeration	Existing active users are rejected before any new database record is created or outbound email is sent.
Session Cleanup	Abandoned or partial registration attempts using the same email are auto-deleted when a user restarts Step 1.
Protected Views	Steps 2 and 3 enforce state verification via <code>request.session.get("verifying_email")</code> . Direct URL traversal without a prior session is redirected back to Step 1.
State Isolation	The <code>verifying_email</code> session variable is removed via <code>del request.session["verifying_email"]</code> immediately upon successful activation, preventing state reuse or replay.

7. Production Hardening Notes

The current implementation is functional but includes a few placeholders that should be addressed before a production deployment:

- Password hashing — `activate_password_view` currently stores `raw_password` directly into `password_hash`. Replace with Django's `make_password()` (or migrate to Django's built-in `User/AbstractUser` model) before going live.
- OTP expiry — the `otp_code` field has no time-to-live. Add an `otp_generated_at` timestamp and enforce a short expiry window (5–10 minutes) to limit brute-force exposure.
- OTP rate limiting — add throttling on `verify_otp_view` to prevent repeated guesses against a fixed 6-digit code.
- Email deliverability — confirm `DEFAULT_FROM_EMAIL` and SMTP backend settings are production-ready, and handle send failures gracefully.
- HTTPS & secure cookies — ensure `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, and `SECURE_SSL_REDIRECT` are enabled in `settings.py` for any deployment beyond local development.

— End of Document — LoginSystem v1.0 —